HACKTOBERFEST / ISLAMABAD / 2026

PARTICIPANT PROCESS MANUAL

DATE	VENUE	BUILD WINDOW
10 October 2026	COWORK24 · I-10	2 hours 30 minutes

What this manual covers

The practical path from check-in to submission, the mandatory project requirements, the documentation judges need, the MLH submission fields, judging criteria, and how to work with mentors.

PROCESS • REQUIREMENTS • DOCUMENTATION • SUBMISSION • JUDGING • MENTORING

01 / PARTICIPANT FLOW

The process

Follow the same sequence whether you run your model locally or use an eligible cloud API.

STEP	ACTION	WHAT TO DO
01	CHECK-IN	Arrive and confirm your event registration with the organizers.
02	LOGIN / REGISTER	Open the designated MLH submission platform or complete the required account setup.
03	CLAIM AI CREDITS	Follow the organizer instructions for any event credits and test access before building.
04	DEFINE THE MVP	Choose a problem, set a small scope, and identify the model and core feature.
05	BUILD & TEST	Implement the working product, test the main flow, and keep secrets out of source control.
06	PUBLISH ON GITHUB	Create a public repository with source code, README, license, and useful technical diagrams.
07	SUBMIT ON MLH	Enter the project title, description, and public GitHub repository URL on the MLH platform.
08	REVIEW & DEMO	Organizers review eligibility; teams demonstrate their project and answer judge questions.

Build small, demonstrate clearly

Prioritize one complete user journey over extra features. A functional, well-documented prototype is more useful to judges than an unfinished ambitious build.

Team size: 1–3 participants. The event's announced submission deadline and the specific MLH submission link should be followed.

02 / MANDATORY CONDITIONS

Project requirements

Every submission must meet the applicable requirements below.

AI model and API

- Maximum model size: 8 billion parameters (8B) for both local models and cloud-hosted models.
- Participants may use existing models locally through Ollama, Hugging Face tools, LM Studio, or other suitable runtimes, or use a cloud API such as Groq.
- Disclose the exact model name, parameter size, runtime or API provider, and how the model is integrated into the project.
- Use open-source or open-weight AI meaningfully in the project's core functionality; a superficial or incidental AI feature is not sufficient.

Examples: Qwen3-4B via Ollama; Llama 3.1 8B Instant via Groq, if available to the account at build time. Check provider availability before relying on a hosted model.

Reference links: Ollama | Hugging Face | LM Studio | Groq Console | Groq API docs

Originality and open source

- The project must contain meaningful original work by the team.
- The source code must be available in a public GitHub repository.
- Include a recognized open-source license, such as MIT or Apache-2.0, where appropriate.
- Acknowledge significant third-party models, libraries, datasets, tools, and existing projects; respect their licenses.
- A tutorial copy, cosmetic-only fork, or renamed project without meaningful contribution does not qualify as original work.

Repository, security, and demonstrability

- Include clear setup, run, and test instructions, plus the required model details.
- Do not publish API keys, passwords, access tokens, private participant data, or other secrets.
- The project should have a functional core feature that can be demonstrated and explained to judges.
- For an Agent Skill, follow the applicable Agent Skill Open Standard. For a model harness, document the original implementation or meaningful changes.

03 / REPOSITORY GUIDANCE

README, findings & diagrams

The README should let someone understand the product, run it, and see what was actually tested.

README content

SECTION	WHAT TO INCLUDE
Project and problem	Project name, one-line summary, target user, problem, solution, and core features.
AI component	Exact model name, parameter size, runtime/API provider, and what the model does.
Setup and usage	Dependencies, installation commands, configuration, environment-variable names, run instructions, and a minimal usage example.
Testing and findings	Test scenario, input, expected output, actual outcome, pass/fail status, observed issues, and known limitations.
Team and licensing	Team member names and contributions, license, and acknowledgements for third-party components.

Diagrams to include when relevant

DIAGRAM	WHAT IT SHOULD EXPLAIN
System architecture	The main components (for example UI, backend, data store, and AI model/API) and how they connect.
Data Flow Diagram (DFD)	How information enters the system, passes through processes or AI components, touches any data store, and reaches the user.
Sequence / workflow diagram	The order of events in an important interaction, such as sending input to the backend, calling a model, and returning the result.
ER diagram	Use when the project has a relational or structured database; show entities and their relationships.

Use diagrams that match the implementation; keep Mermaid diagrams in README.md accurate as the architecture changes.

Short Mermaid examples

System architecture: show the user interface, application logic, and eligible model/API. Rename the components to match your actual project.

```mermaid
flowchart LR
  U[User] --> UI[App / UI]
  UI --> B[Backend / workflow]
  B --> M[Eligible AI model / API]
  M --> B
  B --> U
```

Data flow: show how user input is processed and returned. Add a database node only if your project uses one.

```mermaid
flowchart LR
  U[User input] --> P((Main process))
  P --> M[AI model / API]
  M --> P
  P --> O[Output to user]
```

04 / SUBMIT YOUR PROJECT

MLH submission guidelines

The designated MLH platform is the official submission channel for this challenge.

Submit the following fields through the event's MLH submission page. Organizers will share the event-specific submission link and deadline.

FIELD	WHAT TO ENTER
Title	The project's name. Keep it clear, specific, and consistent with the GitHub repository.
Description	A concise explanation of the problem, what the product does, the main feature, and how eligible AI is used.
GitHub Link	The full URL of the public repository. Confirm that it opens without sign-in and includes the README and license.

What judges should find in the repository

The MLH form uses the title, description, and repository link. Put the implementation details in the repository README: model/provider details, setup and test instructions, architecture or data-flow diagrams, examples or screenshots, actual test findings, limitations, team contributions, license, and acknowledgements.

Submission deadline

Use the cutoff time announced by the organizers. The event-specific MLH submission URL is not specified in this manual; use the link shared by the organizers.

Before submitting

Review the project title and description for accuracy, verify the GitHub URL, check public access, confirm the README renders correctly, and ensure no credentials or private data are committed.

05 / EVALUATION

Judging criteria

Judges assess the implementation, the contribution of AI, originality, usefulness, and the quality of the open-source submission.

CRITERION	WEIGHT	WHAT JUDGES LOOK FOR
Technical implementation	30%	Does the project work? Is the engineering meaningful and well implemented?
Meaningful open-source/open-weight AI usage	25%	Is the eligible AI model central to the project rather than a superficial extra?
Originality and creativity	20%	Is the idea interesting, distinctive, or meaningfully different?
Usefulness and impact	15%	Does it address a real problem or provide practical value to users?
Open-source quality and demo	10%	Is the repository licensed and documented, and can the team demonstrate what it built?

Total weighting

100% across the five criteria above.

Demo expectations

Show the main user flow working, explain the problem and intended users, identify the exact AI model and its role, describe the team's original contribution, and answer questions about testing, limitations, and implementation choices.

The original challenge brief presents these weights as a recommended framework and notes that organizers may adjust them before judging. The final rubric is the one confirmed by the organizers.

06 / DURING THE BUILD

Mentoring & participant conduct

Mentors and organizers are there to help teams move past blockers and make progress during the build sprint.

Mentoring

The challenge brief states that mentors and organizers will circulate during the build sprint and help teams. It does not define a formal mentor, follow the instructions provided at the venue.

To make a mentoring conversation useful, explain what you are trying to achieve, show the relevant code or error, describe what you already tried, and ask one focused question. Share a minimal reproducible example where possible. Apply advice carefully and make sure your team understands the changes it submits.

Teamwork and etiquette

- Treat other participants, mentors, organizers, and judges respectfully. Share space and time fairly during help sessions.
- Be transparent about individual contributions and the use of existing code, models, libraries, and AI-assisted development tools.
- Follow venue and organizer instructions. Do not misuse event credits, accounts, API access, or another participant's resources.
- Do not claim work you did not create, misrepresent model usage, or hide known limitations that affect the demo.
- Protect credentials and private data, and report exposed secrets or security concerns to the organizers promptly.

> **The goal**
> Build a useful working product, document it honestly, publish the code openly, and be ready to explain the decisions your team made.